

UI Folders by User Role & Access

Views

- **admin:** The master control dashboard for shop owners or general managers.
 - **employees / hr:** Portals for managing staff profiles, attendance, or payroll.
 - **customers:** The client-facing views where car owners can check their status.
 - **technician / helper:** Simple interfaces for mechanics to view assigned repair jobs.
 - **gm:** Specialized high-level reports for the General Manager.
-

UI Folders by DMS Feature

- **workorders / workorder-transactions:** The core screens for creating repair tickets and logging job history.
 - **parts / pricing:** Screens for managing inventory stocks, ordering parts, and setting labor markup rates.
 - **vehicles:** The database views tracking license plates, VINs, and car models.
 - **transactions / approvals / stm:** Pages handling cash receipts, service approvals, and status tracking management.
 - **auth:** The login, registration, and password reset screens.
-

Structural Layout Layout

- **partials:** Reusable UI blocks like your navigation bars, headers, and footers.
- **index.ejs:** The main landing page or core template entry point.

Data

data.json: This is your active database. It holds your live records, like customer profiles, car logs, and active repair tickets.

🔍 **data.backup...json files:** These are automatic historical backups created before critical system changes (like migrating user access roles or cleaning up technician records).

🔍 **store.js:** This is the script file containing the code that reads from and writes data to data.json.

Lib

In an auto shop DMS, **comeback-metrics.js** tracks **warranty reworks** or **repeat repairs**.

Node_modules

System & Core Utilities

- **.bin:** Holds the executable command-line scripts for your developer tools.
 - **async:** Provides powerful functions for handling asynchronous JavaScript operations sequentially.
 - **balanced-match:** Matches sections of text inside brackets or parentheses.
 - **bytes:** Utility for parsing and formatting raw byte strings (e.g., converting 1024 to 1KB).
-

Request & File Handling

- **accepts:** Detects what type of data formats (like JSON or HTML) a user's browser can accept.
 - **body-parser:** Extracts incoming data from submitted forms so your routes can read it.
 - **content-disposition:** Instructs the browser whether to display a file or download it as an attachment.
 - **content-type:** Handles the headers that tell the browser what type of data is being sent.
 - **csv-parser:** Parses flat-rate labor and auto parts data out of your VehServiceLabor.csv file.
-

Security & Cookies

- **cookie:** Generates and parses browser cookies to track user sessions.
 - **cookie-signature:** Signs and verifies cookies to prevent users from tampering with their session data.
-

File System Monitoring

- **anymatch:** Matches file paths against specific rules or strings.
 - **binary-extensions:** Returns a list of known binary file extensions (like .png or .exe).
 - **chokidar:** Monitors your local files for real-time changes to restart your server automatically during development.
-

Internal Code Bindings

- **array-flatten:** Flattens nested arrays of data into a single flat list.
 - **brace-expansion:** Expands comma-separated shell patterns inside text strings.
 - **braces:** Fast, comprehensive glob matching tool for paths.
 - **call-bind-apply-helpers / call-bound:** Internal utility functions used by deeper Node.js packages to manage execution context.
 - **concat-map:** Runs a map function over a collection and joins the results together.
 - **debug:** A tiny JavaScript debugging utility that prints color-coded logs to your terminal window.
 - **depd:** Warns you in the console if your code uses outdated, deprecated features.
-

This image shows your public folder expanded, along with your main backend architectural folders. These control what your users see and interact with in your DMS application.

Inside the public Folder

- **icons/**
 - A directory containing small UI images and asset graphics.
 - Stores visual buttons for repairs, parts, and print features.
 - **styles.css**
 - The main design document for your application.
 - Controls the visual look, button styling, colors, and layout layout of your shop dashboards.
-

Application Structural Folders

- **routes/**
 - Houses the traffic directors of your app.
 - Maps web URLs to the backend functions that calculate parts markup and labor hours.
- **scripts/**
 - Contains system automation utilities.
 - Runs backend tasks like updating flat-rate prices or migrating historical service records.
- **views/**
 - Holds your user-facing layout templates (EJS files).
 - Generates the screens that mechanics, service advisors, and customers see.

This image shows your **routes directory**. These files are the backend traffic controllers for your DMS. They listen for web requests (like clicking a button or opening a page) and execute the logic for that specific part of your auto shop business. Here is exactly what each file handles:

routes

- **admin.js:** Controls master settings, system overrides, and high-level configuration tools for the shop owner.
- **employees.js:** Handles HR logic like adding new mechanics, tracking staff profiles, and logging clock-in times.

- **helper.js:** Manages restricted access paths or simple functions built specifically for shop helpers or apprentices.

Core Workshop Operations

- **vehicles.js:** Handles requests to add, search, or update customer cars, including license plates, VINs, and service histories.
- **workorders.js:** Directs the core lifecycle of a repair ticket, from creating a new job to tracking its progress.
- **workorder-transactions.js:** Tracks itemized entries inside a ticket, like adding a specific line item for a brake pad replacement.
- **technician.js:** Houses the routes for the mechanic's interface, allowing them to mark jobs complete or view assigned tickets.

Parts & Finance

- **parts.js:** Directs your physical parts inventory management, including stock counts, reordering, and shelf locations.
- **pricing.js:** Calculates repair margins, flat-rate matrix sheets, and automatic markups on shop supplies or parts.
- **transactions.js:** Processes invoice closures, cash drawer receipts, and final customer billings.

Metrics & Business Logic

- **approvals.js:** Handles digital or verbal service authorizations when a customer approves or declines an inspection estimate.
- **auth.js:** Manages secure system entry, including user login sessions, registrations, and logouts for all roles.
- **customers.js:** Manages client contact profiles, loyalty data, and their specific service appointment scheduling.
- **kpi.js:** Aggregates business performance data, tracking metrics like daily sales, car count, and technician efficiency.

💡 High-Utility Token Strategy

Because these files are split up cleanly by feature, you can easily save tokens. If you are fixing a bug with parts pricing, **only type @pricing.js** into your Cursor chat. Never add the entire @routes folder, as that would force the AI to read all 15 files at once.

This image displays your **scripts directory**. Unlike your routes, these are **utility scripts** run manually in the terminal by a developer or automatically via cron jobs. They handle data maintenance, history cleanup, and simulations for your shop.

Here is what each file does:

Scripts

Data Cleanup & Standardization

- **backfill-transactions.js**
 - Populates missing financial details in older invoices.
 - Ensures legacy repair jobs have complete accounting parameters.
- **normalize-technician-identities.js**
 - Cleans up messy or duplicate technician records.

- Standardizes mechanic names (e.g., changing "Dave S." and "David Smith" to a unique ID).
 - **normalize-work-order-numbers.js**
 - Standardizes the formatting structure of your repair tickets.
 - Formats existing tickets into a uniform style (e.g., converting ticket #45 to WO-00045).
-

Imports, Backups, & Diagnostics

- **backup-app.js**
 - Automates archival snapshots of your core application settings.
 - Prevents catastrophic loss before running database adjustments.
 - **import-transaction-records.js**
 - Imports large batches of historical sales from legacy shop software.
 - Parses outside ledger datasets directly into your local database.
 - **refresh-transaction-snapshot.js**
 - Rebuilds and syncs cached transaction reports.
 - Updates your financial dashboard snapshots with live data values.
-

Testing & Simulation

- **generate-july-2026-simulation.js**
 - Generates realistic, fake automotive shop data for July 2026.
 - Fills your local database with mock repairs, parts usage, and invoices.
 - Allows you to stress-test your KPI reports and finance algorithms without risking live business metrics.
-

💡 High-Utility Token Strategy

These script files are great targets for your `.cursorignore` file **if** you are done running them. If you don't plan on altering them, add `scripts/` to `.cursorignore` to prevent Cursor from accidentally pulling historical database logic into your general coding chats.

This image shows your expanded **views directory**, along with the remaining files at the very bottom of your project root.

Since your project uses the **EJS template engine**, these folders contain the actual HTML templates that build the frontend screens for your users.

views

UI Subfolders inside views/ Management & Staff Roles

- **admin**: Screens for the shop owner to configure system-wide rules and global labor settings.
- **gm**: High-level reporting dashboards built specifically for the General Manager to track shop profitability.
- **hr**: Human Resources interfaces for onboarding staff, adjusting wages, and tracking attendance logs.
- **employees**: Personal portal views for general staff members to see their profile details.
- **technician**: A streamlined, mobile-friendly interface for mechanics to view assigned cars and update checklist items.
- **helper**: Restricted, simplified views for shop assistants or apprentices to look up basic tasks.

Front Desk & Operations

- **customers**: Screens for managing client profiles, contact information, and vehicle ownership links.
- **vehicles**: Dashboards dedicated to car data, tracking VINs, license plates, models, and service histories.

- **workorders:** The primary screens for creating, editing, and viewing active auto repair tickets.
- **workorder-transactions:** Detailed line-item sub-views for adding specific parts or labor charges to a ticket.

Parts & Financials

- **parts:** Inventory management screens for tracking warehouse stock, shelf locations, and low-parts alerts.
- **pricing:** Tools for setting up pricing matrices, shop supplies surcharges, and parts markup tiers.
- **transactions:** Final billing screens, invoice generation layout, and payment processing portals.

System Utilities & Security

- **auth:** User security screens, including the shop login page, registration forms, and password resets.
- **approvals:** Digital authorization portals where a customer can sign off on estimated repairs via phone or email.
- **stm: Service Tracking Management** screens that show a live status board of cars currently in the bays.
- **partials:** Reusable code snippets embedded across all pages, such as headers, side navigation bars, and footers.

Root Template & Project Files (Bottom)

- **index.ejs**
 - The primary frontend landing page or entry template that coordinates your main view layouts.
- **.cursorignore**
 - The configuration file we created to block heavy data folders from wasting your AI token budget.
- **.env.example**
 - A template blueprint file showing other developers what environment configuration keys (like database paths or ports) are required to run the app.
- **app-admin.js**
 - The core backend server configuration file that boots up the admin and operational portals of the application.
- **app-gm.js**
 - A separate backend server file tailored specifically to launch the reporting engine and server instances for the General Manager dashboard.



Final Token-Saving Reminder

When updating user interfaces, use **@views/folder-name** or **@file-name.ejs** in your prompt instead of letting the AI scan the entire views root directory.

Heavy Data Reports & CSV Sheets

- **employee-db-all.csv**
 - A spreadsheet storing complete background data for your entire auto shop workforce.
 - Contains payroll rates, mechanical certifications, and historical staff logs.
 - **transaction-database.csv**
 - A flat-file dataset compiling every financial action processed by the shop.
 - Tracks individual cash drawer balances, parts invoices, and credit transactions.
 - **VehicleType.csv**
 - A lookup spreadsheet holding car classifications.
 - Used by the system to match vehicle makes and models with correct repair bays.
 - **VehServiceLabor.csv**
 - Your core flat-rate automotive database containing standardized repair labor hours.
 - Used to look up how much time a specific job (like a water pump replacement) should bill out.
-

Node.js Manifest Documents

- **package.json**
 - The central configuration blueprint document for your entire application.
 - Outlines script execution shortcuts and tracks your third-party package dependencies.
 - **package-lock.json**
 - An automated lockfile that registers the absolute exact versions of your dependencies.
 - Guarantees that the app executes flawlessly without library mismatches across separate developer machines.
-

Documentation & Incomplete Downloads

- **SECURE-OPERATIONS.md**
 - A markdown text file defining the shop's security protocols and backend access credentials.
 - Acts as a manual for developers to safely deploy or update the server without exposing data.
 - **Service_Advisor_Detailed_Report.rtf**
 - A Rich Text Format text file serving as a document template or static export.
 - Likely used by service advisors to print out detailed multi-point checkup overviews for clients.
 - **Unconfirmed 599575.crdownload**
 - An incomplete broken download file left over by Google Chrome inside your directory.
 - Safe to delete entirely from your project, as it contains corrupted partial data.
-

CRITICAL TOKEN WARNING: Update .cursorignore Immediately

Files like **transaction-database.csv** and **employee-db-all.csv** are absolute token hogs. If you don't block them, Cursor will pay to read thousands of rows of data line-by-line during basic chats.

Open your **.cursorignore** file right now and ensure it includes these additions:

text

Block final remaining large data tables

employee-db-all.csv

transaction-database.csv

VehicleType.csv

*.crdownload